miniproject

March 23, 2026

1 Dependencies

[ ]:
```
import tqdm as notebook_tqdm
from datasets import load_dataset
from dotenv import load_dotenv
from typing import Callable, Any
from pathlib import Path
import torch as T
import numpy as np
import string
import os
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import umap
import pandas as pd
import time
```

2 Setup

[2]:
```
# Change print width
T.set_printoptions(linewidth=1000)
np.set_printoptions(linewidth=1000)
```

3 Data

Importing the data

[3]:
```
env_path = os.path.join(os.getcwd(), ".env.secrets")
load_dotenv(env_path)

hf_token = os.getenv("HF_TOKEN")

FORMAT = "torch"

ds = load_dataset(
    "Salesforce/wikitext",
```

```
    "wikitext-103-v1",
    token=hf_token if hf_token else False,
).with_format(FORMAT)

x_test, x_train, x_val = (ds[key] for key in ds.keys())

display(x_train["text"][:10])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
```

```
['',
 ' = Valkyria Chronicles III = \n',
 '',
 ' Senjō no Valkyria 3 : <unk> Chronicles ( Japanese :      3 , lit . Valkyria␣
 ↪of the Battlefield 3 ) , commonly referred to as Valkyria Chronicles III␣
 ↪outside Japan , is a tactical role @-@ playing video game developed by Sega␣
 ↪and Media.Vision for the PlayStation Portable . Released in January 2011 in␣
 ↪Japan , it is the third game in the Valkyria series . Employing the same␣
 ↪fusion of tactical and real @-@ time gameplay as its predecessors , the story␣
 ↪runs parallel to the first game and follows the " Nameless " , a penal␣
 ↪military unit serving the nation of Gallia during the Second Europan War who␣
 ↪perform secret black operations and are pitted against the Imperial unit "␣
 ↪<unk> Raven " . \n',
 " The game began development in 2010 , carrying over a large portion of the␣
 ↪work done on Valkyria Chronicles II . While it retained the standard features␣
 ↪of the series , it also underwent multiple adjustments , such as making the␣
 ↪game more forgiving for series newcomers . Character designer <unk> Honjou and␣
 ↪composer Hitoshi Sakimoto both returned from previous entries , along with␣
 ↪Valkyria Chronicles II director Takeshi Ozawa . A large team of writers␣
 ↪handled the script . The game 's opening theme was sung by May 'n . \n",
 " It met with positive sales in Japan , and was praised by both Japanese and␣
 ↪western critics . After release , it received downloadable content , along␣
 ↪with an expanded edition in November of that year . It was also adapted into␣
 ↪manga and an original video animation series . Due to low sales of Valkyria␣
 ↪Chronicles II , Valkyria Chronicles III was not localized , but a fan␣
 ↪translation compatible with the game 's expanded edition was released in 2014 .
 ↪ Media.Vision would return to the franchise with the development of Valkyria :␣
 ↪Azure Revolution for the PlayStation 4 . \n",
 '',
 ' = = Gameplay = = \n',
 '',
```

```
" As with previous <unk> Chronicles games , Valkyria Chronicles III is a␣
↪tactical role @-@ playing game where players take control of a military unit␣
↪and take part in missions against enemy forces . Stories are told through␣
↪comic book @-@ like panels with animated character portraits , with characters␣
↪speaking partially through voiced speech bubbles and partially through␣
↪unvoiced text . The player progresses through a series of linear missions ,␣
↪gradually unlocked as maps that can be freely scanned through and replayed as␣
↪they are unlocked . The route to each story location on the map varies␣
↪depending on an individual player 's approach : when one option is selected ,␣
↪the other is sealed off to the player . Outside missions , the player␣
↪characters rest in a camp , where units can be customized and character growth␣
↪occurs . Alongside the main story missions are character @-@ specific sub␣
↪missions relating to different squad members . After the game 's completion ,␣
↪additional episodes are unlocked , some of them having a higher difficulty␣
↪than those found in the rest of the game . There are also love simulation␣
↪elements related to the game 's two main heroines , although they take a very␣
↪minor role . \n"]
```

4 Setup Training Data

[ ]:
```
class Tokenizer():
    def __init__(self, ds, num_max_sentences, window_size, PAD_TOKEN,␣
 ↪min_count=5):
        self.ds = ds
        self.num_max_sentences = num_max_sentences
        self.window_size = window_size
        self.PAD_TOKEN = PAD_TOKEN
        self.min_count = min_count

        self.vocabulary = self.create_vocabulary(ds)
        self.PAD_INDEX = len(self.vocabulary)

        self.word_to_index = {word: i for i, word in enumerate(self.vocabulary)}
        self.word_to_index[PAD_TOKEN] = self.PAD_INDEX

        self.index_to_word = {i: word for i, word in enumerate(self.vocabulary)}
        self.index_to_word[self.PAD_INDEX] = PAD_TOKEN

        self.VOCAB_SIZE = len(self.vocabulary) + 1
        self.word_freq = self._build_freq()

        print(f"Vocabulary: {self.VOCAB_SIZE} words (min_count={min_count},␣
 ↪sentences={num_max_sentences})")

    def create_vocabulary(self, data):
        """Build vocabulary from training sentences only, keeping words that
        appear at least min_count times. This keeps vocab size proportional
```

```
        to the actual training data rather than the full corpus."""
        to_remove = string.punctuation + "\u201c\u201d\u2018\u2019"
        table = str.maketrans("", "", to_remove)
        freq = {}
        for sentence in data["text"][:self.num_max_sentences]:
            for word in sentence.translate(table).lower().split():
                freq[word] = freq.get(word, 0) + 1
        return sorted(w for w, c in freq.items() if c >= self.min_count)

    def _build_freq(self) -> np.ndarray:
        freq = np.zeros(self.VOCAB_SIZE, dtype=np.float32)
        for sentence in self.ds["text"][:self.num_max_sentences]:
            for w in self.preprocess_sentence(sentence):
                idx = self.word_to_index.get(w)
                if idx is not None:
                    freq[idx] += 1
        return freq

    def get_one_hot_encoding_from_token_id(self, id: int):
        vec = T.zeros(self.VOCAB_SIZE)
        vec[id] = 1
        return vec

    def get_one_hot_encoding(self, word: str):
        vec = T.zeros(self.VOCAB_SIZE)
        vec[self.word_to_index[word]] = 1
        return vec

    def get_vector_one_hot_encodings(self, words: list[str]):
        return [self.get_one_hot_encoding(word) for word in words]

    def preprocess_sentence(self, sentence: str) -> list[str]:
        to_remove = string.punctuation + "\u201c\u201d\u2018\u2019"
        table = str.maketrans("", "", to_remove)
        return sentence.translate(table).lower().split()

    def generate_CBOW_pairs(self, add_padding=False, start=0.0, end=1.0):
        """Generator that yields (context_tensor, target_id) one pair at a time.
        Words absent from the vocabulary (below min_count) are skipped.
        start/end are fractions of num_max_sentences, used for train/val␣
↪splitting."""
        start_idx = int(start * self.num_max_sentences)
        end_idx   = int(end   * self.num_max_sentences)
        for sentence in self.ds["text"][start_idx:end_idx]:
            if len(sentence) < 1:
                continue
            # Only keep words that made it into the vocabulary
```

```
        ids = [self.word_to_index[w]
               for w in self.preprocess_sentence(sentence)
               if w in self.word_to_index]
        for center in range(len(ids)):
            context = []
            for offset in range(-self.window_size, self.window_size + 1):
                if offset == 0:
                    continue
                pos = center + offset
                if 0 <= pos < len(ids):
                    context.append(ids[pos])

            if len(context) == 0:
                continue

            if add_padding and len(context) < (2 * self.window_size):
                dn = 2 * self.window_size - len(context)
                context = [self.PAD_INDEX] * dn + context

            yield (T.tensor(context), ids[center])

def create_CBOW_pairs(
        self,
        add_padding=False,
        one_hot_encoded_context=False,
        one_hot_encoded_target=False
    ):
    pairs = []
    for sentence in (self.ds["text"][:self.num_max_sentences]):
        if len(sentence) < 1:
            continue
        ids = [self.word_to_index[w]
               for w in self.preprocess_sentence(sentence)
               if w in self.word_to_index]
        for center in range(len(ids)):
            context = []
            for offset in range(-self.window_size, self.window_size + 1):
                if offset == 0:
                    continue
                pos = center + offset
                if 0 <= pos < len(ids):
                    context.append(ids[pos])

            if len(context) == 0:
                continue

            if add_padding and len(context) < (2 * self.window_size):
```

```
                    dn = 2 * self.window_size - len(context)
                    context = [self.PAD_INDEX] * dn + context
                target = ids[center]

                if one_hot_encoded_context:
                    context = [self.get_one_hot_encoding_from_token_id(token)␣
 ↪for token in context]
                else:
                    context = T.tensor(context)

                if one_hot_encoded_target:
                    target = self.get_one_hot_encoding_from_token_id(target)

                pairs.append((context, target))
        return pairs
```

[ ]:
```
# Small corpus tokenizer - used for the computationally-expensive model␣
 ↪comparisons
# (softmax CBOW and unoptimized NS CBOW) where a small vocabulary is needed to
# keep runtimes manageable.
SMALL_TOK_FN    = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪tokenizer_small.pt"
_small_tok_path = Path(SMALL_TOK_FN)
max_sentences = 200
window_size   = 2
PAD_TOKEN     = "<PAD>"

if _small_tok_path.exists():
    tokenizer_small = T.load(_small_tok_path, weights_only=False)
    _stale_small = (
        tokenizer_small.num_max_sentences != max_sentences
        or getattr(tokenizer_small, "min_count", None) != 3
    )
    if _stale_small:
        print("Rebuilding small tokenizer …")
        tokenizer_small = Tokenizer(x_train, max_sentences, window_size,␣
 ↪PAD_TOKEN, min_count=3)
        T.save(tokenizer_small, _small_tok_path)
else:
    tokenizer_small = Tokenizer(x_train, max_sentences, window_size, PAD_TOKEN,␣
 ↪min_count=3)
    T.save(tokenizer_small, _small_tok_path)
```

[ ]:
```
class DataLoader():
    def __init__(self, data, batch_size, shuffle=False):
        self.data = data
        self.batch_size = batch_size
```

```
        self.shuffle = shuffle
        self._indices = None
        self.num_data_points = len(data)

    def __iter__(self):
        indices = list(range(len(self.data)))
        if self.shuffle:
            np.random.shuffle(indices)
        self._indices = indices
        self._pos = 0
        return self

    def __next__(self):
        if self._pos >= len(self.data):
            raise StopIteration
        batch = [self.data[i] for i in self._indices[self._pos : self._pos +␣
↪self.batch_size]]
        self._pos += self.batch_size
        return batch

    def __len__(self):
        return len(self.data) // self.batch_size


class StreamingDataLoader():
    """DataLoader that generates (context, target) pairs on demand via a␣
↪generator
    function. No list of all pairs is ever held in memory."""

    def __init__(self, pair_fn, batch_size):
        self.pair_fn = pair_fn      # callable: () -> generator of (context,␣
↪target_id)
        self.batch_size = batch_size
        # Count pairs once at construction so len() and num_data_points are␣
↪available
        self.num_data_points = sum(1 for _ in pair_fn())

    def __len__(self):
        return self.num_data_points // self.batch_size

    def __iter__(self):
        batch = []
        for pair in self.pair_fn():
            batch.append(pair)
            if len(batch) == self.batch_size:
                yield batch
                batch = []
```

```
        if batch:   # yield last partial batch
            yield batch
```

```
Train pairs: 11411  |  Val pairs: 4126
```

[ ]:
```
TRAIN_SPLIT = 0.8
# Small dataloaders - used for the showcase/comparison training runs
dl_small_train = StreamingDataLoader(
    lambda: tokenizer_small.generate_CBOW_pairs(add_padding=True,␣
 ↪end=TRAIN_SPLIT),
    batch_size=32,
)
dl_small_val = StreamingDataLoader(
    lambda: tokenizer_small.generate_CBOW_pairs(add_padding=True,␣
 ↪start=TRAIN_SPLIT),
    batch_size=32,
)
print(
    f"Small corpus  - vocab: {tokenizer_small.VOCAB_SIZE:>5}  "
    f"train pairs: {dl_small_train.num_data_points:>6}  "
    f"val pairs: {dl_small_val.num_data_points:>5}"
)
```

5 Creating the modeling tools

[ ]:
```
class Layer():
    def forward(self, v: Any) -> T.Tensor:
        raise NotImplementedError(f"{type(self).__name__} has not implemented␣
 ↪forward")
```

[ ]:
```
class Derivable(Layer):
    """A layer that can compute gradients but has no trainable weights (e.g.␣
 ↪activations)."""

    def __init__(self):
        super().__init__()
        self.dLdv: T.Tensor = None  # gradient of loss w.r.t. this layer's input

    def backward(self, dLdy: T.Tensor):
        """Compute and store dLdv given upstream gradient dLdy."""
        raise NotImplementedError(f"{type(self).__name__} has not implemented␣
 ↪backward")
```

[ ]:
```
class Trainable(Derivable):
    """A layer with trainable weights supporting mini-batch SGD.

    Mini-batch training protocol:
```

```
    1. model.zero_grad()                        - clear accumulators (once per␣
↪batch)
    2. model.accumulate_gradients(dLdy)         - per sample, inside the batch␣
↪loop
    3. model.backpropogate(batch_size)          - apply weight update (once␣
↪per batch)
   """

   def __init__(self, lr):
       super().__init__()
       self.lr:   float    = lr
       self.dLdw: T.Tensor = None  # gradient of loss w.r.t. weights
       self.dLdv: T.Tensor = None  # gradient of loss w.r.t. input

   def zero_grad(self) -> None:
       """Clear gradient accumulators before a new batch."""
       raise NotImplementedError(f"{type(self).__name__} has not implemented␣
↪zero_grad")

   def accumulate_gradients(self, dLdy: T.Tensor) -> None:
       """Compute gradients for one sample and add to accumulators.
       Sets dLdv for the chain rule. Does NOT update weights."""
       raise NotImplementedError(f"{type(self).__name__} has not implemented␣
↪accumulate_gradients")

   def update_weights(self, batch_size: int) -> None:
       """Apply accumulated gradients divided by batch_size, then reset␣
↪accumulators.
       Called once per batch after all accumulate_gradients calls."""
       raise NotImplementedError(f"{type(self).__name__} has not implemented␣
↪update_weights")
```

5.1 One-hot-encoder

Assigns each word in a vocabulary a n dimensional vector with a single element set to one (uniquely) where the words are sorted alphabetically and n is the number of unique words

[163]:
```
class OneHotEncoder(Layer):
    def __init__(self, vocabulary_size: int):
        self.dim = vocabulary_size

    def forward(self, token: int) -> T.Tensor:
        vec = T.zeros(self.dim)
        vec[token] = 1
        return vec
```

5.2 Vectorized One-hot-encoder

[164]:
```
class OneHotEncoderVectorized(Layer):
    def __init__(self, encoder: OneHotEncoder):
        self.encoder = encoder

    def forward(self, tokens: list[int]) -> T.Tensor:
        return T.stack([self.encoder.forward(token) for token in tokens])
```

[ ]:
```
FN = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/one_hot_encoder.
↪pt"
file_path = Path(FN)
if file_path.exists():
    encoder_small = T.load(file_path, weights_only=False)
    if encoder_small.dim != tokenizer_small.VOCAB_SIZE:
        print(f"Encoder stale (dim {encoder_small.dim} → {tokenizer_small.
↪VOCAB_SIZE}), rebuilding …")
        encoder_small = OneHotEncoder(tokenizer_small.VOCAB_SIZE)
        T.save(encoder_small, file_path)
else:
    encoder_small = OneHotEncoder(tokenizer_small.VOCAB_SIZE)
    T.save(encoder_small, file_path)

print(f"Encoder (small):  dim={encoder_small.dim}")
```

5.3 Softmax

[234]:
```
class Softmax(Derivable):
    def __init__(self):
        super().__init__()
        self._cached_output = None

    def forward(self, z: T.Tensor) -> T.Tensor:
        # Exponentiate all elements of z, z are the logits, or the "raw" input.
        z_exp = T.exp(z)
        # Squeeze away the last element of y, this makes no difference␣
↪mathematically.
        y = (z_exp / T.sum(z_exp, dim=0)).squeeze()

        self._cached_output = y
        return y

    def compute_gradients(self, dLdy: T.Tensor):
        return (self._cached_output * (dLdy - T.dot(self._cached_output, dLdy)))

    def backward(self, dLdy: T.Tensor):
        self.dLdv = self.compute_gradients(dLdy)
```

```
sm = Softmax()
test_values = [
    T.tensor([1.0, 0.0, 0.0, 0.0]).reshape(-1, 1),
    T.tensor([1.0, 1.0, 0.0, 0.0]).reshape(-1, 1),
    T.tensor([1.0, 1.0, 1.0, 0.0]).reshape(-1, 1),
    T.tensor([1.0, 1.0, 1.0, 1.0]).reshape(-1, 1),
]

for v in test_values:
    out = sm.forward(v)
    print(
        f"""Output:\n{out.numpy()},
        """
    )
```

```
Output:
[0.4753669 0.1748777 0.1748777 0.1748777],

Output:
[0.3655293  0.3655293  0.13447072 0.13447072],

Output:
[0.29692274 0.29692274 0.29692274 0.10923178],

Output:
[0.25 0.25 0.25 0.25],
```

5.4 Linear layer

[ ]:
```
class LinearLayer2D(Trainable):
    def __init__(self, dim_in, dim_out, activation, lr,␣
 ↪is_layer_in_average=False):
        super().__init__(lr=lr)
        self.dim_in  = dim_in
        self.dim_out = dim_out
        self.weights = T.randn(dim_in + 1, dim_out) * 0.01
        self.activation = activation
        self.is_layer_in_average = is_layer_in_average
        self._cached_input = []
        self._dLdw_accum   = T.zeros(dim_in + 1, dim_out)  # gradient␣
 ↪accumulator

    def __add_bias(self, v):
        return T.cat((v, T.ones((1,))), dim=0)
```

```
    def forward(self, v):
        vec_plus_bias = self.__add_bias(v)
        self._cached_input.append(vec_plus_bias)
        z = vec_plus_bias @ self.weights
        return self.activation.forward(z).squeeze() if self.activation else z.
↪squeeze()

    def accumulate_gradients(self, dLdy):
        """Compute gradient for this sample and accumulate. Does NOT update␣
↪weights."""
        if not self._cached_input:
            raise RuntimeError("Perform a forward pass before accumulating␣
↪gradients")
        if self.activation is not None:
            self.activation.backward(dLdy)
            dLdz = self.activation.dLdv
        else:
            dLdz = dLdy
        cached_input = self._cached_input.pop()
        self.dLdv          = (self.weights @ dLdz)[:-1]    # strip bias row
        self._dLdw_accum  += T.outer(cached_input, dLdz)

    def zero_grad(self):
        self._dLdw_accum.zero_()

    def update_weights(self, batch_size):
        """Apply accumulated gradient and reset. Call once per batch."""
        self.weights      -= self.lr * self._dLdw_accum / batch_size
        self._dLdw_accum.zero_()


test_values = [T.ones((10,), dtype=T.float32)]
ll = LinearLayer2D(10, 3, Softmax(), 1e-3)
for v in test_values:
    out = ll.forward(v)
    print(f"Input: {v.numpy()}\nOutput: {out.numpy()}\n")
```

```
Input: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
Output: [0.34039378 0.33844656 0.3211597 ]
```

5.5 Linear layer average mixer

This object is used to average the the result of many layers of the same type.

[ ]:
```
class LinearLayerAvgMixer(Trainable):

    def __init__(self, n: int, layer: LinearLayer2D, lr=None):
```

```
        super().__init__(lr)
        self.n     = n
        self.layer = layer

    def forward(self, v: T.Tensor) -> T.Tensor:
        avg = T.zeros(self.layer.dim_out)
        for i in range(self.n):
            avg += self.layer.forward(v[i]) / self.n
        return avg

    def zero_grad(self) -> None:
        self.layer.zero_grad()

    def accumulate_gradients(self, dLdy: T.Tensor) -> None:
        for _ in range(self.n):
            self.layer.accumulate_gradients(dLdy / self.n)
        self.dLdv = self.layer.dLdv

    def update_weights(self, batch_size: int) -> None:
        self.layer.update_weights(batch_size)
```

5.6 Embedding Average Layer (Optimized Lookup)

The `LinearLayerAvgMixer` averages context embeddings by: 1. Encoding each context word index as a one-hot vector $e_i \in \mathbb{R}^V$ 2. Multiplying by the weight matrix: $e_i W = W_{[i]}$ (row i of W) 3. Averaging the results

Step 1 and 2 together are equivalent to a direct row lookup, since multiplying a one-hot vector by a matrix simply selects the corresponding row:

$$\frac{1}{n}\sum_{k=1}^{n} e_{i_k} W = \frac{1}{n}\sum_{k=1}^{n} W_{[i_k]}$$

`EmbeddingAverageLayer` implements this directly without large matrix multiplication and memory costly allocations. It is a mathematically equivalent simplification used to speed up training.

[ ]:
```
class EmbeddingAverageLayer(Trainable):
    """
    Optimized replacement for OneHotEncoderVectorized + LinearLayerAvgMixer.

    Mathematically equivalent to averaging one-hot context vectors multiplied␣
↪by W,
    but replaces the (vocab_size x embed_dim) matmul with a direct row lookup:

        (1/n) * sum(e_{i_k} @ W)  ==  (1/n) * sum(W[i_k])

    This avoids allocating large one-hot vectors (vocab_size-dim) entirely.
    """
```

```
    def __init__(self, vocab_size: int, embed_dim: int, lr: float = 1e-3):
        super().__init__(lr=lr)
        self.vocab_size      = vocab_size
        self.embed_dim       = embed_dim
        self.weights         = T.randn(vocab_size, embed_dim) * 0.1
        self._cached_indices = None
        self._grad_accum     = {}   # idx (int) -> accumulated gradient tensor

    def forward(self, indices: T.Tensor) -> T.Tensor:
        self._cached_indices = indices
        return self.weights[indices].mean(dim=0)   # (embed_dim,)

    def zero_grad(self) -> None:
        self._grad_accum.clear()

    def accumulate_gradients(self, dLdy: T.Tensor) -> None:
        """Accumulate gradient contribution from one sample. Does NOT update␣
↪weights."""
        n    = len(self._cached_indices)
        grad = dLdy / n
        for idx in self._cached_indices:
            idx = int(idx)
            if idx in self._grad_accum:
                self._grad_accum[idx] += grad
            else:
                self._grad_accum[idx] = grad.clone()
        self.dLdv = None   # no differentiable layer upstream

    def update_weights(self, batch_size: int) -> None:
        """Apply accumulated gradients averaged over the batch, then reset."""
        for idx, grad in self._grad_accum.items():
            self.weights[idx] -= self.lr * grad / batch_size
        self._grad_accum.clear()
```

5.7 The Model object

This object holds all the layers and activation functions we have defined, it represents the implementation of the full function that produces and outputs $\hat{\mathbf{y}}$ from an input $\mathbf{v}$.

[ ]:
```
class Model:
    def __init__(self, layers: list[Trainable | Derivable | object],␣
↪debug_mode=False):
        self.layers     = layers
        self.debug_mode = debug_mode

    def forward(self, v: T.Tensor):
```

```
        curr = v
        for l in self.layers:
            if self.debug_mode:
                print(f"Input to {type(l).__name__}: shape {curr.shape if␣
↪isinstance(curr, T.Tensor) else '?'}")
            curr = l.forward(curr)
            if self.debug_mode:
                print(f"Output from {type(l).__name__}: shape {curr.shape}")
        return curr

    def zero_grad(self) -> None:
        """Clear all gradient accumulators. Call once before each batch."""
        for l in self.layers:
            if isinstance(l, Trainable):
                l.zero_grad()

    def accumulate_gradients(self, dLdy: T.Tensor) -> None:
        """Chain the gradient backward through all layers for one sample.
        Calls accumulate_gradients on each Trainable layer"""
        for l in reversed(self.layers):
            if isinstance(l, Trainable):
                l.accumulate_gradients(dLdy)
            if not isinstance(l, OneHotEncoderVectorized):
                dLdy = l.dLdv

    def backpropogate(self, batch_size: int) -> None:
        """Apply accumulated gradients to all layers. Call once per batch."""
        for l in self.layers:
            if isinstance(l, Trainable):
                l.update_weights(batch_size)

    def clear_caches(self) -> None:
        """Discard cached forward-pass inputs after a validation forward pass.
↪"""
        for l in self.layers:
            if hasattr(l, '_cached_input'):
                l._cached_input.clear()
            if hasattr(l, 'layer') and hasattr(l.layer, '_cached_input'):
                l.layer._cached_input.clear()
```

5.8 Binary cross entropy loss

[238]:
```
class BinaryCrossEntropyLoss():
    def __init__(self):
        self.loss = None
        self.dLdy = None
```

```
    def compute_loss(self, y_true: T.Tensor, y_hat: T.Tensor):
        # Log of 0 gives inf. Clamping to avoid crashes
        y_hat = T.clamp(y_hat, min=1e-7, max=1 - 1e-7)

        loss = -(y_true @ T.log(y_hat) + (1-y_true) @ (T.log(1-y_hat)))
        self.loss = loss

        dLdy_hat = ((-y_true) / y_hat) + ((1-y_true) / (1-y_hat))
        self.dLdy = dLdy_hat
        return loss
```

5.9 Negative Sampling Loss

[ ]:
```
class NegativeSamplingLoss():
    def __init__(self, tokenizer, embed_dim, k=5, lr=1e-3, power=0.75):
        self.k              = k
        self.lr             = lr
        self.vocab_size     = tokenizer.VOCAB_SIZE
        self.out_embeddings = T.randn(self.vocab_size, embed_dim) * 0.1
        self.noise_dist     = self._build_noise_dist(tokenizer.word_freq, power)
        self._cached_h          = None
        self._cached_pos_id     = None
        self._cached_neg_ids    = None
        self._cached_pos_score  = None
        self._cached_neg_scores = None
        self.dLdv               = None
        self._pos_grad_accum    = {}   # pos_id -> accumulated gradient
        self._neg_grad_accum    = {}   # neg_id -> accumulated gradient

    def _build_noise_dist(self, word_freq, power):
        smoothed = word_freq ** power
        smoothed += 1e-10
        return smoothed / smoothed.sum()

    def _sample_negatives(self, exclude_id):
        samples = []
        while len(samples) < self.k:
            c = int(np.random.choice(self.vocab_size, p=self.noise_dist))
            if c != exclude_id:
                samples.append(c)
        return samples

    def compute_loss(self, h, pos_id):
        pos_vec   = self.out_embeddings[pos_id]
        pos_score = T.sigmoid(T.dot(h, pos_vec))
        pos_loss  = T.log(pos_score + 1e-7)
```

```
        neg_ids    = self._sample_negatives(pos_id)
        neg_scores = [T.sigmoid(T.dot(h, self.out_embeddings[i])) for i in␣
↪neg_ids]
        neg_loss   = sum(T.log(1 - s + 1e-7) for s in neg_scores)

        self._cached_h          = h
        self._cached_pos_id     = pos_id
        self._cached_neg_ids    = neg_ids
        self._cached_pos_score  = pos_score
        self._cached_neg_scores = neg_scores
        return -(pos_loss + neg_loss)

    def zero_grad(self) -> None:
        self._pos_grad_accum.clear()
        self._neg_grad_accum.clear()

    def backward(self) -> None:
        """Compute dLdv for chain rule and accumulate output embedding␣
↪gradients.
        Does NOT update weights"""
        h         = self._cached_h
        pos_id    = self._cached_pos_id
        pos_score = self._cached_pos_score

        # Gradient w.r.t. h - passed back to input embedding layer via dLdv
        dLdh = (pos_score - 1) * self.out_embeddings[pos_id].clone()

        pos_grad = (pos_score - 1) * h
        if pos_id in self._pos_grad_accum:
            self._pos_grad_accum[pos_id] += pos_grad
        else:
            self._pos_grad_accum[pos_id] = pos_grad.clone()

        for neg_id, neg_score in zip(self._cached_neg_ids, self.
↪_cached_neg_scores):
            dLdh += neg_score * self.out_embeddings[neg_id].clone()
            neg_grad = neg_score * h
            if neg_id in self._neg_grad_accum:
                self._neg_grad_accum[neg_id] += neg_grad
            else:
                self._neg_grad_accum[neg_id] = neg_grad.clone()

        self.dLdv = dLdh

    def update_weights(self, batch_size: int) -> None:
        """Apply accumulated gradients averaged over the batch, then reset."""
```

```
        for idx, grad in self._pos_grad_accum.items():
            self.out_embeddings[idx] -= self.lr * grad / batch_size
        for idx, grad in self._neg_grad_accum.items():
            self.out_embeddings[idx] -= self.lr * grad / batch_size
        self._pos_grad_accum.clear()
        self._neg_grad_accum.clear()
```

6 Creating the Word2Vec model

[ ]:
```
# Softmax CBOW - trained on the small corpus
# Full softmax requires a dense (LATENT_DIM × VOCAB_SIZE) output projection and
# computes a probability distribution over the ENTIRE vocabulary on every␣
 ↪forward
# and backward pass: O(V) cost per sample.  This becomes the dominant bottleneck
# as V grows

INPUT_DIM  = encoder_small.dim   # = tokenizer_small.VOCAB_SIZE
OUTPUT_DIM = encoder_small.dim
LATENT_DIM = 10

word2vecCBOW = Model(
    [
        OneHotEncoderVectorized(encoder=encoder_small),
        LinearLayerAvgMixer(
            n=4,
            layer=LinearLayer2D(
                dim_in=INPUT_DIM,
                dim_out=LATENT_DIM,
                activation=None,
                lr=1e-3,
                is_layer_in_average=True,
            ),
        ),
        LinearLayer2D(
            dim_in=LATENT_DIM,
            dim_out=OUTPUT_DIM,
            activation=Softmax(),
            lr=1e-3,
            is_layer_in_average=False,
        ),
    ],
    debug_mode=False,
)

_n_params_softmax = INPUT_DIM * LATENT_DIM + LATENT_DIM * OUTPUT_DIM
print(f"Softmax CBOW | vocab: {INPUT_DIM} | latent: {LATENT_DIM}")
```

```
print(f"  Trainable params : {_n_params_softmax:,}")
print(f"  Output cost/step : O(V={OUTPUT_DIM}) - full softmax normalisation")
```

6.1 Negative Sampling Word2Vec

[ ]:
```
# Unoptimized NS CBOW - trained on the small corpus
# Negative sampling replaces the O(V) softmax output with O(k) comparisons, but
# the INPUT side still uses full one-hot context vectors and a dense
# (V × LATENT_DIM) matrix multiply.  That input projection is also O(V) per
# context word, so the speedup over softmax is limited to the output layer only.
# The EmbeddingAverageLayer (fast model) eliminates this by replacing the dense
# matmul with a direct O(1) row lookup.

INPUT_DIM   = encoder_small.dim   # = tokenizer_small.VOCAB_SIZE
LATENT_DIM  = 10
K_NEGATIVES = 5

word2vecCBOW_ns = Model(
    [
        OneHotEncoderVectorized(encoder=encoder_small),
        LinearLayerAvgMixer(
            n=4,
            layer=LinearLayer2D(
                dim_in=INPUT_DIM,
                dim_out=LATENT_DIM,
                activation=None,
                lr=1e-3,
                is_layer_in_average=True,
            ),
        ),
    ],
    debug_mode=False,
)

ns_loss = NegativeSamplingLoss(
    tokenizer=tokenizer_small,
    embed_dim=LATENT_DIM,
    k=K_NEGATIVES,
    lr=1e-3,
)

_n_params_ns = INPUT_DIM * LATENT_DIM + tokenizer_small.VOCAB_SIZE * LATENT_DIM
print(f"NS CBOW (unoptimized) | vocab: {INPUT_DIM} | latent: {LATENT_DIM} |␣
↪k={K_NEGATIVES}")
print(f"  Trainable params  : {_n_params_ns:,}")
print(f"  Input cost/step   : O(V={INPUT_DIM}) - dense one-hot × weight matrix")
print(f"  Output cost/step  : O(k={K_NEGATIVES}) - negative sampling")
```

7 Training, Validation, Testing

7.1 Training harness

Two different training loops are required for binary cross entropy loss and negative sampling loss respectively since the negative sampling loss has trainable weights while not fitting explicitly into the model definition. Therefore, the update is done inside the training loop instead if as part of the model backpropogation method. This could be improved as part of future work.

[ ]:
```
def train(model, dataloader, val_dataloader, loss_fn, tokenizer, n_epochs):
    train_losses = []
    val_losses   = []

    for epoch in range(n_epochs):
        # --- Training ---
        epoch_train_loss = 0
        pbar = notebook_tqdm.tqdm(dataloader, total=len(dataloader),␣
↪desc=f"Epoch {epoch+1}/{n_epochs}", leave=True)
        for batch in pbar:
            model.zero_grad()
            batch_loss = 0

            for (context, target) in batch:
                target_one_hot = tokenizer.
↪get_one_hot_encoding_from_token_id(int(target))
                y_hat = model.forward(context)
                loss  = loss_fn.compute_loss(target_one_hot, y_hat)
                batch_loss       += loss
                model.accumulate_gradients(loss_fn.dLdy)    # accumulate per␣
↪sample

            model.backpropogate(len(batch))                 # weight update once␣
↪per batch
            epoch_train_loss += batch_loss
            pbar.set_postfix({"train_loss": f"{float(epoch_train_loss):.4f}"})

        epoch_train_loss /= dataloader.num_data_points
        train_losses.append(epoch_train_loss)

        # --- Validation (no weight updates) ---
        epoch_val_loss = 0
        for batch in val_dataloader:
            for (context, target) in batch:
                target_one_hot = tokenizer.
↪get_one_hot_encoding_from_token_id(int(target))
                y_hat = model.forward(context)
                model.clear_caches()
                epoch_val_loss += loss_fn.compute_loss(target_one_hot, y_hat)
```

```
        epoch_val_loss /= val_dataloader.num_data_points
        val_losses.append(epoch_val_loss)

        print(f"Epoch {epoch+1}/{n_epochs}: train={float(epoch_train_loss):.4f}␣
↪ val={float(epoch_val_loss):.4f}")

    return {"train": train_losses, "val": val_losses}
```

[ ]:
```
def train_ns(model, ns_loss, dataloader, val_dataloader, n_epochs):
    train_losses = []
    val_losses   = []

    for epoch in range(n_epochs):
        # --- Training ---
        epoch_train_loss = 0
        pbar = notebook_tqdm.tqdm(dataloader, total=len(dataloader),␣
↪desc=f"Epoch {epoch+1}/{n_epochs}", leave=True)
        for batch in pbar:
            model.zero_grad()
            ns_loss.zero_grad()
            batch_loss = 0

            for (context, target_id) in batch:
                target_id = int(target_id)
                h    = model.forward(context)
                loss = ns_loss.compute_loss(h, target_id)
                batch_loss += loss.item()
                ns_loss.backward()                          # accumulate per␣
↪sample
                model.accumulate_gradients(ns_loss.dLdv)   # accumulate per␣
↪sample

            ns_loss.update_weights(len(batch))              # weight update␣
↪once per batch
            model.backpropogate(len(batch))                 # weight update␣
↪once per batch

            epoch_train_loss += batch_loss
            pbar.set_postfix({"train_loss": f"{epoch_train_loss:.4f}"})

        epoch_train_loss /= dataloader.num_data_points
        train_losses.append(epoch_train_loss)

        # --- Validation (no weight updates) ---
        epoch_val_loss = 0
        for batch in val_dataloader:
```

```
            for (context, target_id) in batch:
                target_id = int(target_id)
                h     = model.forward(context)
                model.clear_caches()
                epoch_val_loss += ns_loss.compute_loss(h, target_id).item()

        epoch_val_loss /= val_dataloader.num_data_points
        val_losses.append(epoch_val_loss)

        print(f"Epoch {epoch+1}/{n_epochs}: train={epoch_train_loss:.4f} ␣
↪val={epoch_val_loss:.4f}")

    return {"train": train_losses, "val": val_losses}
```

7.2 Continous bag of words

[ ]:
```
ce = BinaryCrossEntropyLoss()

_t0 = time.perf_counter()
softmax_history = train(
    word2vecCBOW, dl_small_train, dl_small_val, ce, tokenizer_small, n_epochs=5
)
_softmax_time = time.perf_counter() - _t0
print(f"\nSoftmax CBOW - 5 epochs | {_softmax_time:.1f}s total  "
      f"({_softmax_time / dl_small_train.num_data_points / 5 * 1000:.2f} ms/
↪sample)")
```

```
Epoch 1/5: 176it [00:02, 87.35it/s, train_loss=41114.2188]

Epoch 1/5: train=7.3157  val=7.3182

Epoch 2/5: 176it [00:02, 83.40it/s, train_loss=41095.5859]

Epoch 2/5: train=7.3124  val=7.3151

Epoch 3/5: 176it [00:02, 85.59it/s, train_loss=41076.9609]

Epoch 3/5: train=7.3091  val=7.3121

Epoch 4/5: 176it [00:02, 84.63it/s, train_loss=41058.3672]

Epoch 4/5: train=7.3058  val=7.3090

Epoch 5/5: 176it [00:02, 82.90it/s, train_loss=41039.7734]

Epoch 5/5: train=7.3025  val=7.3060

Softmax CBOW - 5 epochs | 11.2s total  (0.40 ms/sample)
```

[ ]:
```
_t0 = time.perf_counter()
ns_history = train_ns(
    word2vecCBOW_ns, ns_loss, dl_small_train, dl_small_val, n_epochs=5
```

```
)
_ns_time = time.perf_counter() - _t0
print(f"\nNS CBOW (unoptimized) - 5 epochs | {_ns_time:.1f}s total  "
      f"({_ns_time / dl_small_train.num_data_points / 5 * 1000:.2f} ms/sample)")
```

7.3 Optimized Word2Vec with EmbeddingAverageLayer

The model below uses `EmbeddingAverageLayer` instead of `OneHotEncoderVectorized + LinearLayerAvgMixer`. The architecture is identical in terms of what it learns where the input is now the raw integer context tensor rather than one-hot vectors.

[ ]:
```
LATENT_DIM_FAST  = 50
K_NEGATIVES_FAST = 5
LR               = 0.2
N_EPOCHS         = 10

embedding_layer = EmbeddingAverageLayer(
    vocab_size=tokenizer_small.VOCAB_SIZE,
    embed_dim=LATENT_DIM_FAST,
    lr=LR
)
word2vecCBOW_fast = Model([embedding_layer], debug_mode=False)

ns_loss = NegativeSamplingLoss(
    tokenizer=tokenizer_small,
    embed_dim=LATENT_DIM_FAST,
    k=K_NEGATIVES_FAST,
    lr=LR
)

_t0 = time.perf_counter()
ns_fast_history = train_ns(
    word2vecCBOW_fast, ns_loss, dl_small_train, dl_small_val, n_epochs=5
)
_ns_optimized_time = time.perf_counter() - _t0
print(f"\nNS CBOW (optimized) - 5 epochs | {_ns_optimized_time:.1f}s total  "
      f"({_ns_optimized_time / dl_small_train.num_data_points / 5 * 1000:.2f}␣
 ↪ms/sample)")
```

```
Epoch 1/5: 176it [00:02, 69.16it/s, train_loss=23351.6172]

Epoch 1/5: train=4.1551  val=4.1504

Epoch 2/5: 176it [00:02, 70.42it/s, train_loss=23233.6503]

Epoch 2/5: train=4.1341  val=4.1202

Epoch 3/5: 176it [00:02, 72.88it/s, train_loss=22962.8649]

Epoch 3/5: train=4.0859  val=4.0572
```

```
Epoch 4/5: 176it [00:02, 69.88it/s, train_loss=22409.9073]

Epoch 4/5: train=3.9875  val=3.9675

Epoch 5/5: 176it [00:02, 70.91it/s, train_loss=21680.5903]

Epoch 5/5: train=3.8578  val=3.8233

NS CBOW (optimized) - 5 epochs | 13.7s total  (0.49 ms/sample)
```

[ ]:
```python
# Complexity comparison
V  = tokenizer_small.VOCAB_SIZE
d  = LATENT_DIM
k  = K_NEGATIVES
ms_softmax = _softmax_time / (dl_small_train.num_data_points * 5) * 1000
ms_ns = _ns_time / (dl_small_train.num_data_points * 5) * 1000
ms_ns_optimized = _ns_optimized_time / (dl_small_train.num_data_points * 5) *␣
↪1000

print("=" * 68)
print("  Per-Sample Cost Breakdown (small corpus, 5 epochs)")
print("=" * 68)
print(f"  {'Model':<36}  {'ms/sample':>10}  {'Relative speed':>14}")
print(f"  {' '*36}  {' '*10}  {' '*14}")
print(f"  {'Softmax CBOW':<36}  {ms_softmax:>9.3f}  {'1×':>14}")
print(f"  {'NS CBOW (unoptimized)':<36}  {ms_ns:>9.3f}  {ms_softmax/ms_ns:>13.
↪2f}×")
print(f"  {'NS CBOW (optimized)':<36}  {ms_ns_optimized:>9.3f}  {ms_softmax/
↪ms_ns_optimized:>13.2f}×")
```

```
====================================================================
  Per-Sample Cost Breakdown (small corpus, 5 epochs)
====================================================================
  Model                                  ms/sample  Relative speed

  Softmax CBOW                               0.420              1×
  NS CBOW (unoptimized)                      0.648           0.65×
  NS CBOW (optimized)                        0.486           0.86×
```

7.4 Multi-Scale Training

The same `EmbeddingAverageLayer` + negative-sampling architecture is trained on three corpus sizes to study how data volume affects embedding quality. All variants share the same hyper-parameters (`embed_dim=50`, `k=5` negatives, `lr=0.2`, `batch_size=32`, 10 epochs) so that only the training data volume differs.

Variant	Sentences	min_count	Vocab Size
Small	200	3	~600
Medium	1 000	3	~2600

Variant	Sentences	min_count	Vocab Size
Large	5 000	5	~5 500

Checkpoints are saved after the first run and reloaded on subsequent runs, so re-executing the notebook does not retrain.

[ ]:
```
WEIGHTS_SMALL = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪word2vec_ns_small.pt"
_small_weights_path = Path(WEIGHTS_SMALL)

embedding_layer_small = EmbeddingAverageLayer(
    vocab_size=tokenizer_small.VOCAB_SIZE,
    embed_dim=LATENT_DIM_FAST,
    lr=LR,
)
word2vec_small = Model([embedding_layer_small], debug_mode=False)
ns_loss_small  = NegativeSamplingLoss(
    tokenizer=tokenizer_small,
    embed_dim=LATENT_DIM_FAST,
    k=K_NEGATIVES_FAST,
    lr=LR,
)

if _small_weights_path.exists():
    ckpt = T.load(_small_weights_path, weights_only=False)
    embedding_layer_small.weights = ckpt["input_embeddings"]
    ns_loss_small.out_embeddings  = ckpt["output_embeddings"]
    ns_history_small              = ckpt["history"]
    print(f"Loaded small model  ({len(ns_history_small['train'])} epochs␣
 ↪trained)")
else:
    ns_history_small = train_ns(
        word2vec_small, ns_loss_small, dl_small_train, dl_small_val, N_EPOCHS
    )
    T.save({
        "input_embeddings":  embedding_layer_small.weights,
        "output_embeddings": ns_loss_small.out_embeddings,
        "history":           ns_history_small,
    }, _small_weights_path)
    print(f"Saved small model to {WEIGHTS_SMALL}")
```

```
Loaded small model  (10 epochs trained)
```

[ ]:
```
MEDIUM_TOK_FN    = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪tokenizer_medium.pt"
_medium_tok_path = Path(MEDIUM_TOK_FN)
medium_max_num_sentences = 1000
```

```
if _medium_tok_path.exists():
    tokenizer_medium = T.load(_medium_tok_path, weights_only=False)
    _stale_medium = (
        tokenizer_medium.num_max_sentences != medium_max_num_sentences
        or getattr(tokenizer_medium, "min_count", None) != 3
    )
    if _stale_medium:
        print("Rebuilding medium tokenizer …")
        tokenizer_medium = Tokenizer(x_train, medium_max_num_sentences,␣
 ↪window_size, PAD_TOKEN, min_count=3)
        T.save(tokenizer_medium, _medium_tok_path)
else:
    tokenizer_medium = Tokenizer(x_train, medium_max_num_sentences,␣
 ↪window_size, PAD_TOKEN, min_count=3)
    T.save(tokenizer_medium, _medium_tok_path)

dl_medium_train = StreamingDataLoader(
    lambda: tokenizer_medium.generate_CBOW_pairs(add_padding=True,␣
 ↪end=TRAIN_SPLIT),
    batch_size=32,
)
dl_medium_val = StreamingDataLoader(
    lambda: tokenizer_medium.generate_CBOW_pairs(add_padding=True,␣
 ↪start=TRAIN_SPLIT),
    batch_size=32,
)
print(
    f"Medium corpus | vocab: {tokenizer_medium.VOCAB_SIZE:>2} | "
    f"train pairs: {dl_medium_train.num_data_points:>2} | "
    f"val pairs: {dl_medium_val.num_data_points:>2}"
)
```

```
Rebuilding medium tokenizer …
Vocabulary: 2588 words (min_count=3, sentences=1000)
Medium corpus | vocab: 2588 | train pairs: 31190 | val pairs: 8714
```

[ ]:
```
WEIGHTS_MEDIUM = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪word2vec_ns_medium.pt"
_medium_weights_path = Path(WEIGHTS_MEDIUM)

embedding_layer_medium = EmbeddingAverageLayer(
    vocab_size=tokenizer_medium.VOCAB_SIZE,
    embed_dim=LATENT_DIM_FAST,
    lr=LR,
)
word2vec_medium = Model([embedding_layer_medium], debug_mode=False)
```

```
ns_loss_medium  = NegativeSamplingLoss(
    tokenizer=tokenizer_medium,
    embed_dim=LATENT_DIM_FAST,
    k=K_NEGATIVES_FAST,
    lr=LR,
)

if _medium_weights_path.exists():
    ckpt = T.load(_medium_weights_path, weights_only=False)
    embedding_layer_medium.weights = ckpt["input_embeddings"]
    ns_loss_medium.out_embeddings  = ckpt["output_embeddings"]
    ns_history_medium              = ckpt["history"]
    print(f"Loaded medium model  ({len(ns_history_medium['train'])} epochs␣
 ↪trained)")
else:
    ns_history_medium = train_ns(
        word2vec_medium, ns_loss_medium, dl_medium_train, dl_medium_val,␣
 ↪N_EPOCHS
    )
    T.save({
        "input_embeddings":  embedding_layer_medium.weights,
        "output_embeddings": ns_loss_medium.out_embeddings,
        "history":           ns_history_medium,
    }, _medium_weights_path)
    print(f"Saved medium model to {WEIGHTS_MEDIUM}")
```

```
Loaded medium model  (10 epochs trained)
```

[ ]:
```
LARGE_TOK_FN    = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪tokenizer_large.pt"
_large_tok_path = Path(LARGE_TOK_FN)
large_max_num_sentences = 5000

if _large_tok_path.exists():
    tokenizer_large = T.load(_large_tok_path, weights_only=False)
    _stale_large = (
        tokenizer_large.num_max_sentences != large_max_num_sentences
        or getattr(tokenizer_large, "min_count", None) != 5
    )
    if _stale_large:
        print("Rebuilding large tokenizer …")
        tokenizer_large = Tokenizer(x_train, large_max_num_sentences,␣
 ↪window_size, PAD_TOKEN, min_count=5)
        T.save(tokenizer_large, _large_tok_path)
else:
    tokenizer_large = Tokenizer(x_train, large_max_num_sentences, window_size,␣
 ↪PAD_TOKEN, min_count=5)
```

```
    T.save(tokenizer_large, _large_tok_path)

dl_large_train = StreamingDataLoader(
    lambda: tokenizer_large.generate_CBOW_pairs(add_padding=True,␣
 ↪end=TRAIN_SPLIT),
    batch_size=32,
)
dl_large_val = StreamingDataLoader(
    lambda: tokenizer_large.generate_CBOW_pairs(add_padding=True,␣
 ↪start=TRAIN_SPLIT),
    batch_size=32,
)
print(
    f"Large corpus | vocab: {tokenizer_large.VOCAB_SIZE:>2} | "
    f"train pairs: {dl_large_train.num_data_points:>2} | "
    f"val pairs: {dl_large_val.num_data_points:>2}"
)
```

```
Rebuilding large tokenizer …
Vocabulary: 5576 words (min_count=5, sentences=5000)
Large corpus | vocab: 5576 | train pairs: 172850 | val pairs: 33148
```

[ ]:
```
WEIGHTS_LARGE = "/home/enarde/Documents/Design-Of-AI-Systems/miniproject/
 ↪word2vec_ns_large.pt"
_large_weights_path = Path(WEIGHTS_LARGE)

embedding_layer_large = EmbeddingAverageLayer(
    vocab_size=tokenizer_large.VOCAB_SIZE,
    embed_dim=LATENT_DIM_FAST,
    lr=LR,
)
word2vec_large = Model([embedding_layer_large], debug_mode=False)
ns_loss_large  = NegativeSamplingLoss(
    tokenizer=tokenizer_large,
    embed_dim=LATENT_DIM_FAST,
    k=K_NEGATIVES_FAST,
    lr=LR,
)

if _large_weights_path.exists():
    ckpt = T.load(_large_weights_path, weights_only=False)
    embedding_layer_large.weights = ckpt["input_embeddings"]
    ns_loss_large.out_embeddings  = ckpt["output_embeddings"]
    ns_history_large              = ckpt["history"]
    print(f"Loaded large model  ({len(ns_history_large['train'])} epochs␣
 ↪trained)")
else:
```

```
ns_history_large = train_ns(
    word2vec_large, ns_loss_large, dl_large_train, dl_large_val, N_EPOCHS
)
T.save({
    "input_embeddings":  embedding_layer_large.weights,
    "output_embeddings": ns_loss_large.out_embeddings,
    "history":           ns_history_large,
}, _large_weights_path)
print(f"Saved large model to {WEIGHTS_LARGE}")
```

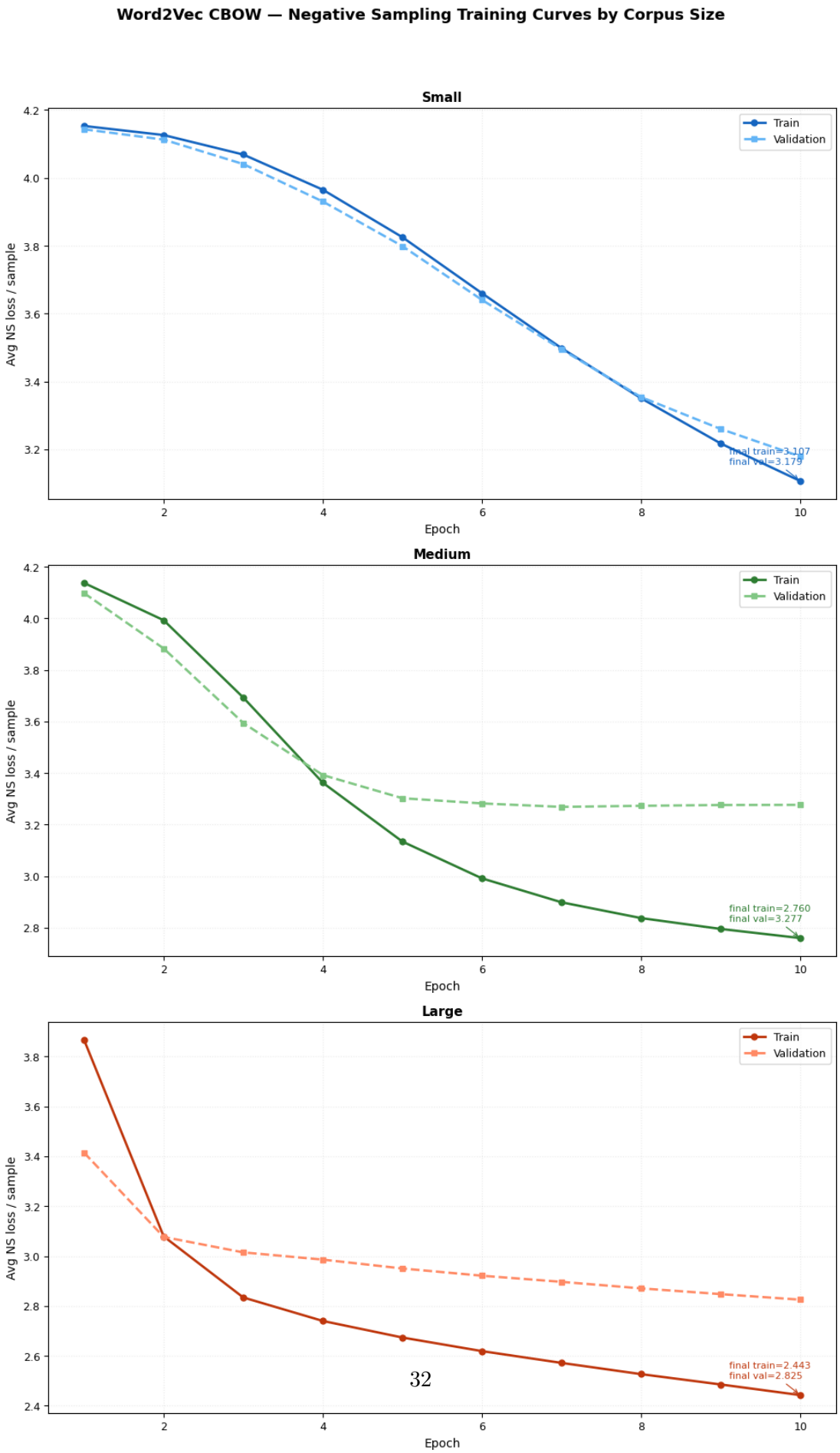
```
Epoch 1/10: 5402it [01:39, 54.06it/s, train_loss=668270.0683]

Epoch 1/10: train=3.8662  val=3.4144

Epoch 2/10: 5402it [01:41, 52.96it/s, train_loss=532166.0391]

Epoch 2/10: train=3.0788  val=3.0767

Epoch 3/10: 5402it [01:42, 52.58it/s, train_loss=489858.1849]

Epoch 3/10: train=2.8340  val=3.0148

Epoch 4/10: 5402it [01:40, 53.91it/s, train_loss=473578.2279]

Epoch 4/10: train=2.7398  val=2.9858

Epoch 5/10: 5402it [01:42, 52.86it/s, train_loss=462101.3629]

Epoch 5/10: train=2.6734  val=2.9502

Epoch 6/10: 5402it [01:44, 51.57it/s, train_loss=452676.5024]

Epoch 6/10: train=2.6189  val=2.9212

Epoch 7/10: 5402it [01:44, 51.75it/s, train_loss=444506.1285]

Epoch 7/10: train=2.5716  val=2.8966

Epoch 8/10: 5402it [01:43, 52.18it/s, train_loss=436723.9294]

Epoch 8/10: train=2.5266  val=2.8704

Epoch 9/10: 5402it [01:43, 52.19it/s, train_loss=429566.5759]

Epoch 9/10: train=2.4852  val=2.8474

Epoch 10/10: 5402it [01:40, 53.70it/s, train_loss=422200.3557]

Epoch 10/10: train=2.4426  val=2.8251
Saved large model to /home/enarde/Documents/Design-Of-AI-
Systems/miniproject/word2vec_ns_large.pt
```

7.5 Visualizing results

7.5.1 Training vs Validation Loss Plot

[ ]:
```
# Combined training-curve comparison
_scale_configs = [
    ("Small",  "ns_history_small"),
    ("Medium", "ns_history_medium"),
    ("Large", "ns_history_large"),
]
_palette = [("#1565C0", "#64B5F6"), ("#2E7D32", "#81C784"), ("#BF360C",␣
 ↪"#FF8A65")]

# Only include variants that have been trained
_histories = []
for label, var_name in _scale_configs:
    try:
        _histories.append((label, eval(var_name)))
    except NameError:
        pass

n_plots = len(_histories)
fig, axes = plt.subplots(n_plots, 1, figsize=(10, 5.5 * n_plots), sharey=False)
if n_plots == 1:
    axes = [axes]

for ax, (label, hist), (c_tr, c_val) in zip(axes, _histories, _palette):
    epochs = range(1, len(hist["train"]) + 1)
    ax.plot(epochs, hist["train"], color=c_tr,  marker="o", lw=2,
            label="Train",      ms=5)
    ax.plot(epochs, hist["val"],   color=c_val, marker="s", lw=2,
            label="Validation", ms=5, linestyle="--")
    ax.set_title(label, fontsize=11, fontweight="bold")
    ax.set_xlabel("Epoch", fontsize=10)
    ax.set_ylabel("Avg NS loss / sample", fontsize=10)
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.25, linestyle=":")
    ax.tick_params(labelsize=9)
    final_tr  = hist["train"][-1]
    final_val = hist["val"][-1]
    ax.annotate(
        f"final train={final_tr:.3f}\nfinal val={final_val:.3f}",
        xy=(len(epochs), final_tr),
        xytext=(-60, 14),
        textcoords="offset points",
        fontsize=8,
        color=c_tr,
        arrowprops=dict(arrowstyle="->", color=c_tr, lw=0.8),
```

```
    )

fig.suptitle(
    "Word2Vec CBOW - Negative Sampling Training Curves by Corpus Size",
    fontsize=13, fontweight="bold", y=1.03,
)
plt.tight_layout()
plt.show()
```


Word2Vec CBOW — Negative Sampling Training Curves by Corpus Size
Small
Train
Validation
final train=3.107
final val=3.179
Avg NS loss / sample
Epoch
Medium
Train
Validation
final train=2.760
final val=3.277
Avg NS loss / sample
Epoch
Large
Train
Validation
final train=2.443
final val=2.825
Avg NS loss / sample
Epoch

7.5.2 K-Nearest Neighbors (Cosine Similarity)

[ ]:
```
def nearest_neighbors(word, embeddings, word_to_index, index_to_word, n=8):
    """Return the n most similar words by cosine similarity."""
    idx = word_to_index.get(word)
    if idx is None:
        return None
    vec    = embeddings[idx]
    norms  = embeddings.norm(dim=1).clamp(min=1e-8)
    scores = (embeddings @ vec) / (norms * vec.norm().clamp(min=1e-8))
    top    = scores.argsort(descending=True)[1 : n + 1]   # skip self at rank 0
    return [(index_to_word[i.item()], round(scores[i].item(), 4)) for i in top]


def neighbors_dataframe(probe_words, embeddings, word_to_index, index_to_word,␣
 ↪n=5):
    """Build a DataFrame with one row per probe word and columns for each rank.
 ↪"""
    rows = []
    for word in probe_words:
        result = nearest_neighbors(word, embeddings, word_to_index,␣
 ↪index_to_word, n=n)
        if result is None:
            continue
        row = {"Query": word}
        for rank, (nbr, score) in enumerate(result, 1):
            row[f"#{rank}"] = f"{nbr}  ({score:.3f})"
        rows.append(row)
    return pd.DataFrame(rows).set_index("Query")


# Probe words grouped by semantic category
probe_words = [
    # time / sequence
    "time", "year", "century", "age",
    # geography / politics
    "world", "city", "war", "government", "american",
    # people / society
    "people", "king", "queen", "church",
    # descriptive
    "new", "great", "first", "history",
]

print("=" * 72)
```

```python
print("  Nearest Neighbours - Large corpus")
print("=" * 72)
df_medium = neighbors_dataframe(
    probe_words,
    embedding_layer_large.weights,
    tokenizer_large.word_to_index,
    tokenizer_large.index_to_word,
    n=8,
)
display(df_medium)

# Cross-scale comparison for a subset of words
_compare_words = ["time", "world", "war", "king", "first"]
_models = [
    ("Small",  embedding_layer_small, tokenizer_small),
    ("Medium",  embedding_layer_medium, tokenizer_medium),
    ("Large", embedding_layer_large, tokenizer_large),
]

print("\n" + "=" * 72)
print("  Cross-Scale Comparison - top-5 neighbours per model")
print("=" * 72)

for query in _compare_words:
    print(f"\n  Query: '{query}'")
    print(f"  {'Model':<22}  Neighbours")
    print(f"  {' '*22}  {' '*45}")
    for model_label, emb_layer, tok in _models:
        result = nearest_neighbors(
            query, emb_layer.weights, tok.word_to_index, tok.index_to_word, n=5
        )
        if result is None:
            print(f"  {model_label:<22}  (word not in vocabulary)")
        else:
            nbrs = ",  ".join(f"{w} ({s:.3f})" for w, s in result)
            print(f"  {model_label:<22}  {nbrs}")
```

```
========================================================================
  Nearest Neighbours - Large corpus
========================================================================

                              #1                  #2                #3  \
Query
time            british  (0.906)       day  (0.893)     same  (0.887)
year               only  (0.885)       but  (0.877)     time  (0.856)
century       following  (0.860)    before  (0.854)    under  (0.845)
age         archaeology  (0.741)      best  (0.728)       ga  (0.727)
world              game  (0.872)   british  (0.853)   second  (0.850)
```

city	first	(0.888)	top	(0.877)	team	(0.864)
war	season	(0.870)	new	(0.866)	british	(0.863)
government	team	(0.856)	gods	(0.839)	song	(0.836)
american	game	(0.896)	season	(0.878)	league	(0.875)
people	an	(0.876)	out	(0.871)	many	(0.865)
king	project	(0.827)	season	(0.753)	team	(0.753)
queen	player	(0.641)	aviation	(0.576)	rays	(0.562)
church	film	(0.856)	album	(0.826)	ship	(0.819)
new	second	(0.912)	first	(0.908)	season	(0.893)
great	side	(0.784)	third	(0.777)	made	(0.773)
first	second	(0.936)	season	(0.917)	new	(0.908)
history	some	(0.879)	first	(0.877)	games	(0.868)

	#4		#5		#6	\
Query						
time	team	(0.883)	first	(0.881)	end	(0.870)
year	new	(0.853)	egyptian	(0.853)	british	(0.850)
century	at	(0.840)	during	(0.829)	against	(0.828)
age	number	(0.722)	four	(0.721)	university	(0.721)
world	early	(0.833)	league	(0.830)	american	(0.827)
city	end	(0.857)	second	(0.851)	british	(0.851)
war	first	(0.860)	all	(0.849)	end	(0.849)
government	army	(0.834)	site	(0.830)	film	(0.826)
american	only	(0.867)	british	(0.867)	national	(0.865)
people	some	(0.859)	their	(0.856)	these	(0.854)
king	rock	(0.752)	building	(0.750)	area	(0.750)
queen	as	(0.560)	for	(0.553)	full	(0.549)
church	second	(0.817)	project	(0.811)	team	(0.807)
new	british	(0.891)	only	(0.890)	best	(0.887)
great	main	(0.767)	club	(0.763)	season	(0.761)
first	team	(0.904)	best	(0.903)	film	(0.896)
history	north	(0.865)	its	(0.863)	all	(0.861)

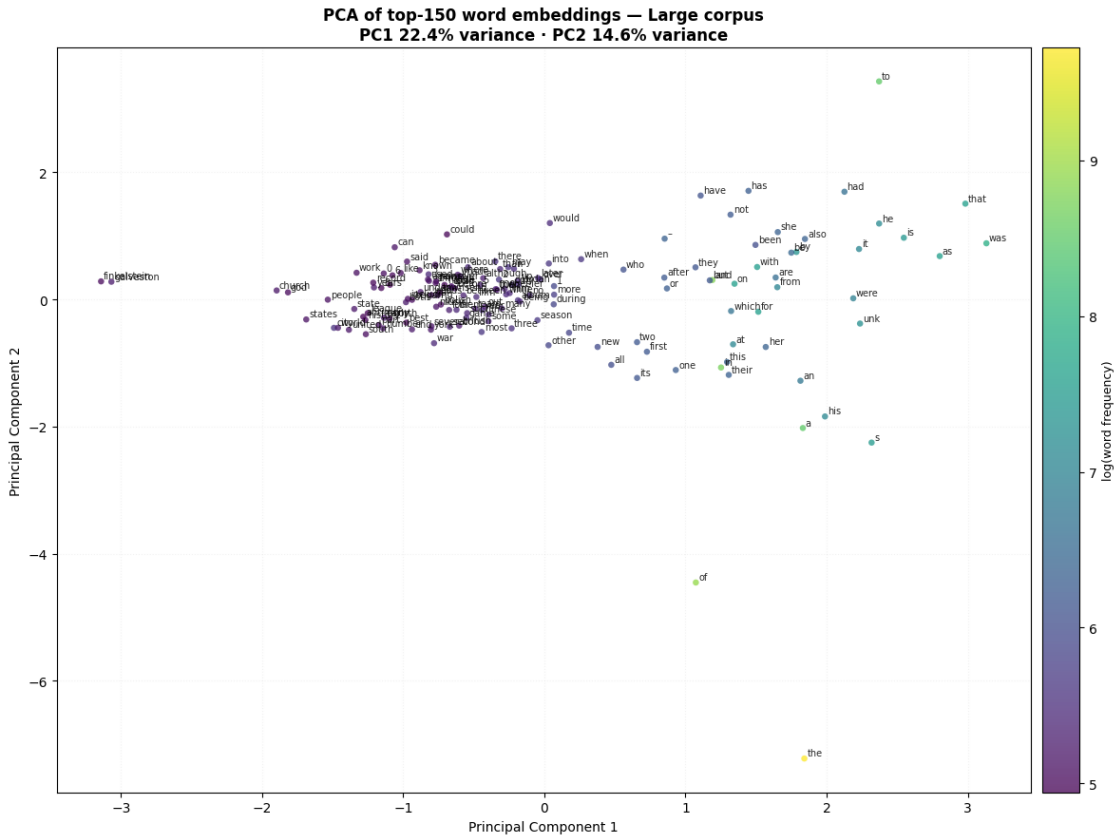
	#7		#8	
Query				
time	north	(0.870)	new	(0.867)
year	made	(0.848)	half	(0.847)
century	from	(0.825)	team	(0.823)
age	out	(0.717)	townsend	(0.716)
world	top	(0.825)	school	(0.822)
city	site	(0.851)	season	(0.848)
war	team	(0.849)	state	(0.846)
government	fleet	(0.826)	album	(0.825)
american	three	(0.863)	north	(0.860)
people	each	(0.851)	all	(0.851)
king	although	(0.747)	before	(0.744)
queen	association	(0.546)	or	(0.541)
church	song	(0.807)	last	(0.806)

```
new                 two  (0.885)       all  (0.883)
great             album  (0.758)    school  (0.755)
first              york  (0.895)       all  (0.894)
history           their  (0.857)       new  (0.852)


======================================================================
  Cross-Scale Comparison - top-5 neighbours per model
======================================================================

  Query: 'time'
  Model                   Neighbours

  Small                   mary (0.398),  available (0.391),  over (0.371),
major (0.354),  removed (0.353)
  Medium                  in (0.817),  at (0.791),  and (0.782),  was (0.782),
<PAD> (0.780)
  Large                   british (0.906),  day (0.893),  same (0.887),  team
(0.883),  first (0.881)

  Query: 'world'
  Model                   Neighbours

  Small                   waldrons (0.415),  1911 (0.410),  has (0.404),  such
(0.399),  form (0.395)
  Medium                  gods (0.749),  game (0.748),  of (0.746),  while
(0.738),  for (0.735)
  Large                   game (0.872),  british (0.853),  second (0.850),
early (0.833),  league (0.830)

  Query: 'war'
  Model                   Neighbours

  Small                   the (0.510),  were (0.503),  and (0.497),  was
(0.493),  rock (0.473)
  Medium                  between (0.643),  only (0.625),  may (0.624),  was
(0.615),  with (0.609)
  Large                   season (0.870),  new (0.866),  british (0.863),  first
(0.860),  all (0.849)

  Query: 'king'
  Model                   Neighbours

  Small                   (word not in vocabulary)
  Medium                  blackie (0.525),  during (0.522),  they (0.498),  to
(0.496),  has (0.488)
  Large                   project (0.827),  season (0.753),  team (0.753),  rock
(0.752),  building (0.750)
```

```
  Query: 'first'
  Model                    Neighbours

  Small                    once (0.488),  by (0.447),  unk (0.432),  that
(0.431),  an (0.422)
  Medium                   for (0.859),  with (0.833),  and (0.832),  is (0.831),
in (0.828)
  Large                    second (0.936),  season (0.917),  new (0.908),  team
(0.904),  best (0.903)
```

[ ]:
```python
import matplotlib.colors as mcolors

TOP_N = 150   # most-frequent words to visualise

def plot_pca_embeddings(embeddings_tensor, tokenizer_obj, top_n=TOP_N,␣
 ↪title_suffix=""):
    freq = tokenizer_obj.word_freq.copy()
    freq[tokenizer_obj.PAD_INDEX] = 0
    top_indices = np.argsort(freq)[::-1][:top_n]
    top_words   = [tokenizer_obj.index_to_word[i] for i in top_indices]
    top_freq    = freq[top_indices]

    vecs_np = embeddings_tensor.detach().numpy()[top_indices]
    pca     = PCA(n_components=2, random_state=42)
    reduced = pca.fit_transform(vecs_np)
    var_exp = pca.explained_variance_ratio_

    # Color by log-frequency quartile
    log_freq  = np.log1p(top_freq)
    norm      = mcolors.Normalize(vmin=log_freq.min(), vmax=log_freq.max())
    cmap      = plt.cm.viridis
    colors    = cmap(norm(log_freq))

    fig, ax = plt.subplots(figsize=(13, 9))
    sc = ax.scatter(
        reduced[:, 0], reduced[:, 1],
        c=log_freq, cmap="viridis", s=22, alpha=0.75, linewidths=0,
    )
    for i, word in enumerate(top_words):
        ax.annotate(
            word,
            (reduced[i, 0], reduced[i, 1]),
            fontsize=7,
            alpha=0.85,
            xytext=(2, 2),
            textcoords="offset points",
```

```
        )

    cbar = plt.colorbar(sc, ax=ax, pad=0.01)
    cbar.set_label("log(word frequency)", fontsize=9)

    ax.set_title(
        f"PCA of top-{top_n} word embeddings{title_suffix}\n"
        f"PC1 {var_exp[0]*100:.1f}% variance · PC2 {var_exp[1]*100:.1f}%␣
↪variance",
        fontsize=12, fontweight="bold",
    )
    ax.set_xlabel("Principal Component 1", fontsize=10)
    ax.set_ylabel("Principal Component 2", fontsize=10)
    ax.grid(True, alpha=0.15, linestyle=":")
    plt.tight_layout()
    plt.show()

plot_pca_embeddings(
    embedding_layer_large.weights,
    tokenizer_large,
    title_suffix=" - Large corpus",
)
```

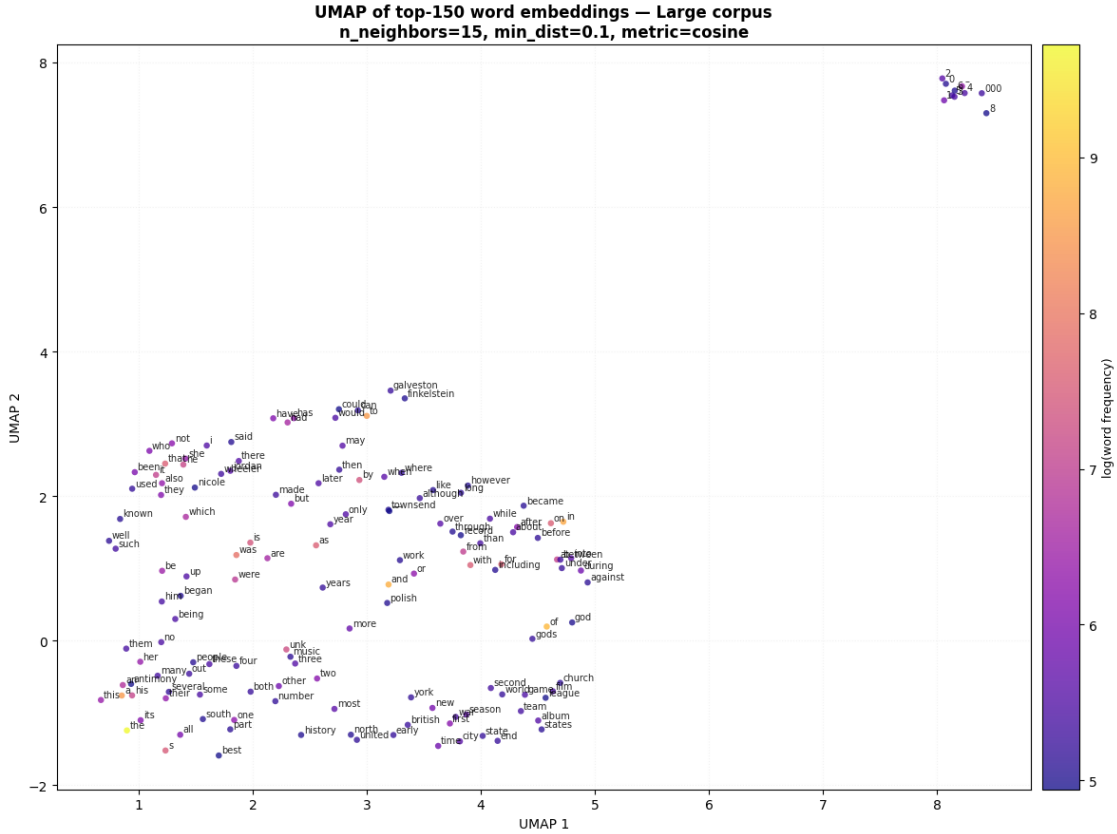


7.5.3 UMAP

[ ]:
```
def plot_umap_embeddings(embeddings_tensor, tokenizer_obj, top_n=150,␣
↪title_suffix="",
                         n_neighbors=15, min_dist=0.1):
    freq = tokenizer_obj.word_freq.copy()
    freq[tokenizer_obj.PAD_INDEX] = 0
    top_indices = np.argsort(freq)[::-1][:top_n]
    top_words   = [tokenizer_obj.index_to_word[i] for i in top_indices]
    top_freq    = freq[top_indices]

    vecs_np = embeddings_tensor.detach().numpy()[top_indices]

    reducer = umap.UMAP(
        n_components=2,
        n_neighbors=n_neighbors,
        min_dist=min_dist,
        random_state=42,
        metric="cosine",
    )
    reduced  = reducer.fit_transform(vecs_np)
    log_freq = np.log1p(top_freq)

    fig, ax = plt.subplots(figsize=(13, 9))
    sc = ax.scatter(
        reduced[:, 0], reduced[:, 1],
        c=log_freq, cmap="plasma", s=22, alpha=0.75, linewidths=0,
    )
    for i, word in enumerate(top_words):
        ax.annotate(
            word,
            (reduced[i, 0], reduced[i, 1]),
            fontsize=7,
            alpha=0.85,
            xytext=(2, 2),
            textcoords="offset points",
        )

    cbar = plt.colorbar(sc, ax=ax, pad=0.01)
    cbar.set_label("log(word frequency)", fontsize=9)

    ax.set_title(
        f"UMAP of top-{top_n} word embeddings{title_suffix}\n"
        f"n_neighbors={n_neighbors}, min_dist={min_dist}, metric=cosine",
```

```
        fontsize=12, fontweight="bold",
    )
    ax.set_xlabel("UMAP 1", fontsize=10)
    ax.set_ylabel("UMAP 2", fontsize=10)
    ax.grid(True, alpha=0.15, linestyle=":")
    plt.tight_layout()
    plt.show()

plot_umap_embeddings(
    embedding_layer_large.weights,
    tokenizer_large,
    title_suffix=" - Large corpus",
)
```

```
/home/enarde/jupyterlab-env/lib/python3.12/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```



[ ]:
```
# Summary comparison across all three trained models
_summary_configs = [
    ("Small",  tokenizer_small,  embedding_layer_small,  "ns_history_small"),
    ("Medium", tokenizer_medium, embedding_layer_medium, "ns_history_medium"),
    ("Large",  tokenizer_large,  embedding_layer_large,  "ns_history_large"),
]

rows = []
for label, tok, emb_layer, hist_var in _summary_configs:
    try:
        hist = eval(hist_var)
    except NameError:
        continue
    n_params = tok.VOCAB_SIZE * emb_layer.embed_dim   # input embeddings only
    rows.append({
        "Model":            label,
        "Vocab size":       tok.VOCAB_SIZE,
        "Embed params":     f"{n_params:,}",
        "Train pairs":      f"{int(tok.num_max_sentences * 0.8 * 20):,}",   #␣
 ↪rough estimate
        "Final train loss": f"{hist['train'][-1]:.4f}",
        "Final val loss":   f"{hist['val'][-1]:.4f}",
        "Epochs":           len(hist["train"]),
    })

df_summary = pd.DataFrame(rows).set_index("Model")
print("=" * 72)
print("  Model Comparison Summary")
print("=" * 72)
display(df_summary)

# PCA side-by-side for all three
_vis_configs = [
    (embedding_layer_small,  tokenizer_small,  "Small"),
    (embedding_layer_medium, tokenizer_medium, "Medium"),
    (embedding_layer_large,  tokenizer_large,  "Large"),
]

fig, axes = plt.subplots(3, 1, figsize=(10, 18))
for ax, (emb_layer, tok, label) in zip(axes, _vis_configs):
    freq = tok.word_freq.copy()
    freq[tok.PAD_INDEX] = 0
    top_n   = min(100, tok.VOCAB_SIZE - 1)
    top_idx = np.argsort(freq)[::-1][:top_n]
    words   = [tok.index_to_word[i] for i in top_idx]
    log_f   = np.log1p(freq[top_idx])
```

```
    vecs    = emb_layer.weights.detach().numpy()[top_idx]
    pca     = PCA(n_components=2, random_state=42)
    red     = pca.fit_transform(vecs)
    var     = pca.explained_variance_ratio_

    sc = ax.scatter(red[:, 0], red[:, 1], c=log_f, cmap="viridis",
                    s=18, alpha=0.75, linewidths=0)
    for i, w in enumerate(words):
        ax.annotate(w, (red[i, 0], red[i, 1]), fontsize=6, alpha=0.8,
                    xytext=(1, 1), textcoords="offset points")
    ax.set_title(
        f"{label}\nPC1 {var[0]*100:.1f}% · PC2 {var[1]*100:.1f}%",
        fontsize=10, fontweight="bold",
    )
    ax.set_xlabel("PC1", fontsize=9)
    ax.set_ylabel("PC2", fontsize=9)
    ax.grid(True, alpha=0.15, linestyle=":")
    plt.colorbar(sc, ax=ax, pad=0.01).set_label("log freq", fontsize=8)

fig.suptitle(
    "PCA Comparison - Word Embeddings Across Corpus Sizes (top-100 words)",
    fontsize=12, fontweight="bold", y=1.02,
)
plt.tight_layout()
plt.show()
```

[ ]:

```

```